Concurrency & Distributed Systems

Week 2 — Part 2: Lifecycle States + Scheduling Myths

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

May 11, 2026

Why This Block Matters

- Threads are **state machines**, not "running things"
- Debugging hangs = reading **state transitions**
- Performance bottlenecks = **BLOCKED** and contention
- Correctness must not depend on **timing**

Engineering lens

Concurrency failures are often **scheduler + lock + state** failures.

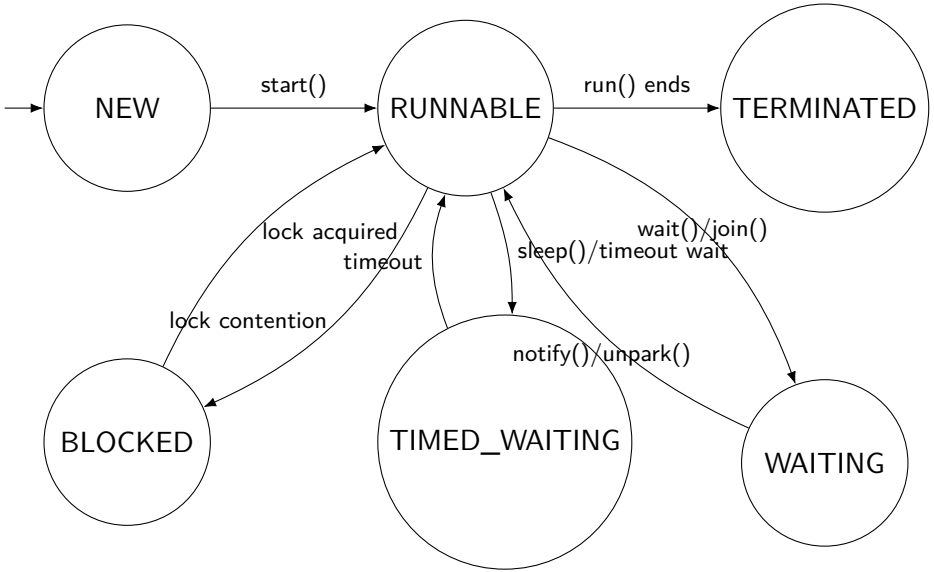
Lifecycle Overview (Java)

- **NEW**
- **RUNNABLE**
- **BLOCKED**
- **WAITING**
- **TIMED_WAITING**
- **TERMINATED**

Key precision

RUNNABLE = **ready OR executing**. Java does not distinguish.

Lifecycle State Diagram



State: NEW

```
Thread t = new Thread(task); // NEW
```

- Java object exists, but thread is not scheduled
- No execution stack is running yet
- Transition happens only with `start()`

Engineer rule

A `Thread` is **single-use**. You cannot `start()` it twice.

State: RUNNABLE

- **Eligible to run** (scheduler can pick it)
- May be:
  - executing on a core
  - waiting in the OS run queue

Critical clarification

RUNNABLE is not “running now” — it is “**can run**”.

State: BLOCKED (Monitor Contention)

```
synchronized(lock) {
        // thread enters only if it owns the monitor
}
```

- BLOCKED = waiting to acquire an intrinsic lock (monitor)
- Symptom of contention and coarse locking

Engineering signal

Many BLOCKED threads usually means **scalability bottleneck**.

BLOCKED vs WAITING (Do Not Confuse)

- **BLOCKED**: trying to enter a synchronized region
- **WAITING**: voluntarily parked for coordination (wait/join)

Mental model

BLOCKED = **contention** WAITING = **coordination**

State: WAITING (Indefinite)

```
// coordination primitives
obj.wait();
t.join();
java.util.concurrent.locks.LockSupport.park();
```

- WAITING until another thread signals
- Typically indicates coordination patterns (good) or missed signals (bad)

State: TIMED_WAITING (Timeout)

```
Thread.sleep(1000);
obj.wait(1000);
t.join(1000);
```

Important

`sleep()` does **not** release locks.

- Timeout means "**at least** this long"
- Wake-up depends on scheduler and system load

State: TERMINATED

- `run()` completed OR uncaught exception occurred
- Stack reclaimed, thread is no longer schedulable
- `Thread` object still exists in heap

Engineering consequence

Thread creation has overhead $\Rightarrow$ thread pools exist.

State Transitions Summary

- NEW → RUNNABLE via `start()`
- RUNNABLE → BLOCKED via lock contention
- RUNNABLE → WAITING via `wait/join/park`
- RUNNABLE → TIMED_WAITING via `sleep`/timeout waits
- RUNNABLE → TERMINATED when `run()` ends

Key observation

Most paths return to **RUNNABLE** because the scheduler is always in control.

Reading Thread Dumps (jstack)

Identifying States in Production

```
"Worker-1" #12 prio=5 os_prio=0 tid=0x00007f...
java.lang.Thread.State: BLOCKED (on object monitor)
at com.example.Service.process(Service.java:42)
- waiting to lock <0x000000076ab90> (owned by "Worker-2")
```

Key Indicators:

- `BLOCKED`: look for `waiting to lock`
- `WAITING`: look for `waiting on condition` / `parked`
- `TIMED_WAITING`: look for `sleeping` / `parking for`

Quick Check: State Transition

Scenario:

A thread calls `Thread.sleep(5000)` while holding a lock.

What state is it in?

1. BLOCKED
2. WAITING
3. TIMED_WAITING
4. RUNNABLE

Answer: 3 (TIMED_WAITING)

Lock is **still held**. Other threads trying to enter the synchronized block will be **BLOCKED**.

Scheduling Myths (What People Believe)

- Threads run in **start order**
- `sleep(1000)` guarantees **exact timing**
- Higher priority threads run **first**
- More threads = more **speed**
- `yield()` forces another thread to run

Reality (What Actually Happens)

- OS scheduler decides
- `sleep` guarantees **minimum** delay
- Priority is a **hint**
- `yield` is a **hint**
- Context switching has overhead
- No fairness guarantee
- Timing-based coordination is fragile and non-portable

Myth: Start Order = Execution Order

False assumption

"I called `t1.start()` before `t2.start()`, so t1 runs first."

- `start()` = eligible, not running
- CPU access is a scheduling decision
- Different runs $\Rightarrow$ different interleavings

Myth: sleep() = Precision

Reality

`sleep(x)` means **at least** `x`, not exactly `x`.

- wake-up depends on system load
- timer resolution varies by OS
- never use sleep for coordination

Myth: Priority Guarantees Order

- Java priority is advisory
- OS may ignore or compress priorities
- results vary across machines and OS

Engineering rule

Priority is not a correctness tool. At best, it is a **performance hint**.

The Real Scheduling Model

- Preemptive scheduling
- Time slicing and run queues
- No strict fairness guarantee
- Context switching costs (CPU + cache)

Looking Ahead: Virtual Threads (Java 21+)

Traditional threads = 1:1 mapping (Java Thread → OS Thread).
Virtual threads = M:N mapping (Many Java Threads → few OS carriers).
Impact: scheduling shifts toward JVM; blocking becomes cheap (conceptually).

Lab: Priority Scheduling Experiment

Goal: Test whether priority reliably determines completion order.

- Run multiple times
- Record observed order
- Compare across runs and machines

Expected outcome

Inconsistent ordering $\Rightarrow$ priority is unreliable for correctness.

Lab Code: Priority Race (Latch Start)

```
import java.util.concurrent.CountDownLatch;
public class PriorityRace {
 public static void main(String[] args) throws Exception {
  for (int run = 1; run <= 10; run++) {
   CountDownLatch start = new CountDownLatch(1);
   Runnable task = () -> {
    try { start.await(); } catch (InterruptedException e) {
     Thread.currentThread().interrupt();
     return;
    }
    Thread t = Thread.currentThread();
    System.out.println("Finished: " + t.getName()
    + " | Priority: " + t.getPriority());
```

.

Lab Code: Priority Race (Latch Start)

```
    Thread low = new Thread(task, "Low");
    Thread med = new Thread(task, "Medium");
    Thread high = new Thread(task, "High");
    low.setPriority(Thread.MIN_PRIORITY);
    med.setPriority(Thread.NORM_PRIORITY);
    high.setPriority(Thread.MAX_PRIORITY);
    low.start(); med.start(); high.start();
    start.countDown(); // release all at once
    low.join(); med.join(); high.join();
    System.out.println("---- Run " + run + " ----\n");
   }
  }
}
```

Why CountDownLatch? Ensures all threads become RUNNABLE before the race begins.

Student Task (Data Collection)

- Run the program 10 times
- Record completion order each run
- Note: does **MAX_PRIORITY** consistently win?

Deliverable

A small table: Run # $\rightarrow$ completion order (Low/Medium/High).

Expected Observation

- Completion order varies across runs
- High priority does not always finish first
- Different OS / hardware may produce different patterns

Conclusion

Priority is an unreliable correctness mechanism.

Engineering Takeaways

- The scheduler decides, not your code
- Priority and yield are hints, not guarantees
- Timing-based coordination is fragile
- Correctness requires explicit synchronization contracts

System-level message

Design as if the scheduler is adversarial.

Block 2 Summary

- Threads move through **states** (finite state machine)
- RUNNABLE = ready + executing
- BLOCKED = lock contention, WAITING = coordination
- Thread dumps reveal truth in production
- Scheduling myths collapse under experimentation

Next: Regaining Control

If we cannot control **time**...

How do we control **access**?

Next block

Synchronization primitives: `synchronized`, locks, atomics, visibility.